Génération et modélisation de surfaces 3d

Ibrahim DOGAN, Juliette CORNU-BESSER
Quentin BERNARD-BOUISSIERES, Daniel DEFOING, Haluk YILMAZ
Publié : 30 avril 2026

Projet d'année INFO-F309
Année Académique 2025-2025

Abstract

Ce projet porte sur l'implémentation d'une application permettant de générer des surfaces en deux et en trois dimensions avec l'algorithme de Perlin. Divers paramètres sont proposés afin de modifier la forme des surfaces produites. Le projet permet de répondre à la problématique lors de simulation digitalisée de différents environnements avec une transition cohérente. La procédure est utilisée notamment dans les jeux vidéos ou lors de simulation de texture d'un objet. Le cœur du projet est développé en C++ à l'aide de la bibliothèque graphique ***OpenFrameworks****. Le choix de cette bibliothèque s'explique pour sa simplicité d'utilisation pour l'implémentation.*

Introduction

Dans le cadre du Printemps des Sciences, ce projet s'inscrit dans une démarche de génération et de modélisation de surfaces tridimensionnelles. L'algorithme choisi ici est celui de Perlin pour répondre à cette démarche. La procédure permet de créer des surfaces cohérentes à partir de valeurs pseudo-aléatoires. Néanmoins, la résolution des surfaces générées peut ne pas être suffisante avec Perlin. Ce document se penche sur l'étude du fonctionnement de l'algorithme de Perlin. Puis il explique sur l'implémentation du programme pour l'application de Perlin. Ensuite, il discute de l'influence des différents paramètres lors de la génération de surfaces en mettant l'accent sur la fréquence. Pour finir, il présente les différentes extensions qui peuvent être apportées sur l'application de Perlin.

Fonctionnement de l'algorithme

Ce chapitre se concentre sur le fonctionnement de l'algorithme de Perlin de Ken Perlin [1].

Fonctionnement de l'algorithme de Perlin

Que ce soit en 2D ou en 3D, l'algorithme de Perlin s'applique sur un ensemble de points répartis dans l'espace. L'objectif est d'attribuer une valeur à chaque point de manière cohérente, en tenant compte de son voisinage, afin d'obtenir un résultat à la fois continu et pseudo-aléatoire.

Cellules Pour introduire cette cohérence locale, l'espace est découpé en cellules virtuelles. Une cellule correspond à un carré en 2D ou un cube en 3D, de côté 1. Chaque point de l'espace appartient à une cellule. À chaque coin de ces cellules est associé un vecteur de gradient. Ce découpage permet de structurer l'espace et de faire en sorte que des points proches soient influencés par des éléments communs.

Vecteurs de gradient Les vecteurs de gradient constituent la source du "bruit". Chaque sommet de cellule possède un vecteur généré de manière pseudo-aléatoire. En 2D, une cellule possède donc 4 vecteurs (un par coin), et en 3D, 8 vecteurs.

Calcul de la valeur de bruit L'algorithme parcourt ensuite chaque point de la surface. Pour simplifier l'explication, nous nous plaçons en 2D. Pour un point donné :

- nous déterminons dans quelle cellule il se trouve
- nous calculons sa position locale à l'intérieur de cette cellule

Ensuite, pour chacun des coins de la cellule :

- nous calculons le vecteur de distance entre le point et le coin
- nous calculons le produit scalaire entre ce vecteur de distance et le vecteur de gradient associé au coin

Nous obtenons ainsi une valeur scalaire par coin (donc 4 en 2D) chaque valeur correspond à l'influence de chaque vecteur de gradient respectif.

Interpolation Ces valeurs sont ensuite interpolées pour obtenir une unique valeur au point considéré. Cette interpolation permet de lisser les transitions entre les cellules. Comme deux points voisins partagent en partie les mêmes vecteurs de gradient, leurs valeurs sont proches. Cela crée une variation progressive et évite les discontinuités.

Implémentation

Algorithme de Perlin

Surface de travail Nous utilisons l'outil d'**OpenFrameWorks**, un `mesh`. Cet outil permet de définir un espace où des points sont définis à la même distance.

Cellules et vecteurs de gradient Les vecteurs de gradient sont initialisés aléatoirement au début de l'algorithme. Ils sont contenus dans une liste. Ensuite, les cellules sont initialisées. Elles contiennent une liste de vecteurs de gradient. Cela permet de représenter les vecteurs de gradient aux sommets de la cellule.

Table de permutations Une table de permutation est initialisée après celle des cellules. Elle contient des indices mélangés aléatoirement en dessous du nombre de vecteurs de gradient uniques. Elle est utilisée lors de la sélection de vecteurs de gradient lors du calcul du bruit. Cela empêche de travailler toujours avec les mêmes vecteurs et évite donc un motif répétitif sur la surface. Cette table apporte une méthode de hachage supplémentaire.

Interpolation Dans sa forme la plus simple, l'interpolation linéaire (ou *linear interpolation*) est définie par :

```
float lerp(float x, float y, float alpha):
    return (1 - alpha) * x + alpha * y
```

L'interpolation se fait progressivement selon chaque axe (par exemple en 2D : d'abord selon x, puis selon y). Le paramètre d'interpolation correspond à la partie fractionnaire des coordonnées du point dans la cellule.

Cependant, dans l'algorithme de Perlin, cette interpolation linéaire est combinée avec une autre fonction, celle de lissage. La fonction de lissage (telle que la fonction *fade*) est appliquée à ces paramètres avant interpolation, afin de garantir une transition douce entre les cellules et éviter les discontinuités visibles. Elle prend la forme du pôlynome, $6t^5 - 15t^4 + 10t^3$.

Paramètres de l'algorithme de Perlin

Amplitude

L'amplitude contrôle l'intensité des variations du bruit de Perlin. En deux dimensions, elle peut être interprétée comme la hauteur maximale d'une surface. C'est un cas d'interprétation du résultat de la valeur de Perlin pour un point. L'amplitude permet de représenter des reliefs en deux dimensions comme des montages.

Rotation des vecteurs de gradient

Un paramètre de rotation peut être appliqué aux vecteurs de gradient, avec des valeurs comprises entre 0 et 360 degrés. Le résultat visuel dépend de la rotation des vecteurs.

Nombre de cellules

Le choix de la taille du tableau de cellules a un impact direct sur l'apparence du bruit généré. Lorsque la taille est petite , le nombre de vecteurs de gradient distincts est limité. Comme ces vecteurs sont réutilisés dans l'espace , des motifs similaires peuvent apparaître régulièrement. Sur une grande surface, cela peut rendre la répétition visible et donner un aspect moins naturel. À l'inverse, lorsque le nombre de cellules est grand, les vecteurs de gradient sont plus variés. Les répétitions existent toujours, mais elles se produisent sur des distances beaucoup plus grandes, ce qui les rend difficilement perceptibles à l'œil. Le bruit paraît alors plus riche et plus naturel.

En pratique, il est donc préférable de choisir une valeur suffisamment grande pour éviter les répétitions visibles, sans pour autant surdimensionner inutilement le tableau.

Valeur seuil

En trois dimensions, nous avons introduit des conditions d'apparition permettant de filtrer ou de moduler les zones où le bruit est visible.

Une condition simple définit un seuil sur la valeur du bruit. Seuls les points dont la valeur dépasse ce seuil sont conservés.

Une valeur de seuil élevée restreint fortement les zones visibles, tandis qu'un seuil plus faible produit une surface plus dense. Pour plus de visibilités, nous utilisons les figures représentées ici 1. Ce type de filtrage est couramment utilisé pour générer des structures discrètes, telles que des îlots ou des cavités.

Fréquence

Avant d'être évaluées par la fonction de bruit, les coordonnées sont multipliées par la fréquence :

$$\texttt{noise}(x \cdot freq,\ y \cdot freq).$$

Ce paramètre influence le nombre de points présents dans une cellule. Lorsque la fréquence est faible, il y a beaucoup de points dans une même cellule. Ces points sont alors influencés par les mêmes vecteurs de gradient, ce qui produit de grandes zones

homogènes. À l'inverse, lorsque la fréquence est élevée, il y a moins de points par cellule. Les points sont donc influencés plus rapidement par des gradients différents, ce qui crée des variations plus fines.

Cas limites Nous étudions différents régimes de la fréquence *freq*, en considérant son effet sur l'évaluation de la fonction de bruit.

Lorsque la fréquence est égale à 1, on évalue directement noise(x,y). Comme les coordonnées du maillage sont entières, les points évalués correspondent aux sommets de la grille interne du bruit de Perlin. À ces positions, les vecteurs de distance sont nuls, donc les produits scalaires sont nuls, et l'interpolation donne une valeur nulle. La surface obtenue est donc uniforme comme visible dans la figure 2a.

Lorsque la fréquence est égale à 0, toutes les coordonnées sont ramenées au même point, généralement (0,0). On évalue donc toujours la même valeur de bruit, ce qui produit également une surface uniforme. La différence avec le cas précédent est qu'ici un seul point est échantillonné, alors que pour une fréquence de 1, on parcourt plusieurs sommets distincts.

Lorsque la fréquence est très proche de 0 sans être nulle, les coordonnées varient très lentement dans l'espace du bruit. Il faut parcourir beaucoup de points du maillage pour changer de cellule. Les points voisins sont donc influencés presque de la même manière, ce qui donne une surface presque uniforme avec de très grandes variations, comme visible dans l'image 3a.

Lorsque la fréquence est un entier supérieur à 1, les coordonnées restent entières après multiplication. On échantillonne donc uniquement des sommets de la grille interne, comme dans le cas *freq*=1, mais en sautant des cellules. Le résultat reste une valeur constante, ce qui produit une surface uniforme.

Enfin, lorsque la fréquence est supérieure à 1 mais non entière, les points se trouvent à l'intérieur des cellules. Cela active l'interpolation et produit des variations. Cependant, si la fréquence est très élevée, les variations deviennent très rapides. Si le maillage n'est pas assez fin, ces variations sont mal représentées, ce qui peut entraîner des artefacts visuels comme visible dans l'image 3b.

Remarque Contrairement à une intuition possible, une augmentation de la fréquence ne réduit pas le nombre de vecteurs de gradient utilisés. Elle augmente en réalité la vitesse de variation du signal dans l'espace, ce qui peut poser des problèmes d'échantillonnage si la résolution est insuffisante.

Extensions

Mouvement brownien fractal

Contrairement à une fonction strictement périodique, le bruit de Perlin est une fonction pseudo-aléatoire continue. Néanmoins, il peut être interprété comme un signal caractérisé par différentes composantes fréquentielles.

Le principe consiste à superposer plusieurs instances de la fonction de bruit, chacune étant évaluée à une fréquence et une amplitude différentes, comme décrit dans la section *Layered Noise* de l'article de Dave Mount [2].

Le nombre de couches superposées est appelé le nombre d'*octaves*. À chaque octave, la fréquence est multipliée par un facteur appelé `lacunarity`, tandis que l'amplitude est réduite selon un facteur appelé `persistence`.

La valeur finale est normalisée par la valeur maximale afin de rester indépendante du nombre d'octaves. Elle correspond à une moyenne pondérée des différentes contributions :

```
for (int i = 0; i < octaves; i++){
    total += noise(vectorCoord * frequence) * amp;
    maxValue += amp;
    frequence *= lacunarity;
    amp *= persistence;
}
return total / maxValue;
```

Cette normalisation garantit que le résultat reste dans un intervalle comparable à celui de la fonction de bruit initiale.

Observation Lorsque le nombre d'octaves augmente comme dans l'image **??**, la surface générée devient plus riche en détails et plus irrégulière. Cela s'explique par l'ajout progressif de composantes de plus en plus haute fréquence.

Cependant, lorsque ces fréquences dépassent la résolution du maillage ou de l'échantillonnage spatial, les détails deviennent trop fins pour être correctement représentés. Nous prenons comme image en appui la figure 5. Ce phénomène peut entraîner des artefacts visuels, similaires à de l'aliasing, donnant une apparence bruitée ou instable à la surface. Pour plus d'explications sur ce phénomène, nous renvoyons à l'artciel de Reynald Arnerin sur la partie *Aliasing*, [3].

Interprétation En pratique, au-delà d'un certain nombre d'octaves, l'ajout de nouvelles couches n'apporte plus de contribution visuellement significative, notamment en raison des limitations de résolution.

Théoriquement, chaque augmentation de fréquence s'accompagne d'une diminution de l'amplitude afin de maintenir un équilibre entre les différentes échelles de détails. Le paramètre `persistence`, compris entre 0 et 1, contrôle cette décroissance de l'amplitude.

Une valeur faible de `persistence` atténue rapidement les hautes fréquences, produisant une surface plus lisse, tandis qu'une valeur plus élevée conserve davantage de détails fins.

Réapplication de Perlin

Dans le prolongement des travaux présentés dans le travail de Franky Hung [4], nous avons choisi de superposer plusieurs applications de bruit de Perlin en trois dimensions, chacune utilisant des paramètres distincts.

Cette approche permet de combiner plusieurs champs de bruit aux caractéristiques différentes, enrichissant ainsi la complexité visuelle du résultat.

Conclusion

Ce travail a permis de présenter le fonctionnement de l'algorithme de Perlin pour la génération de cartes de bruit. De plus, nous avons discuté des choix d'implémentation pour ce projet et les subtilités des outils représentés. Puis, une analyse des paramètres a été proposée, avec une attention particulière portée à la fréquence. Enfin, nous avons abordé ses extensions, notamment à travers l'utilisation des octaves et la superposition de différentes paramétrisations. Dans une perspective d'approfondissement, il serait pertinent d'explorer d'autres algorithmes de génération de bruit.

Annexes

References

[1] Ken Perlin. *Improved Noise reference implementation*. 2002. URL: `https://mrl.cs.nyu.edu/~perlin/noise/` (visited on 03/06/2026).

[2] Dave Mount. *Procedural Generation: Perlin Noise*. 2018. URL: `https://www.cs.umd.edu/class/fall2019/cmsc425/handouts/lect14-perlin.pdf` (visited on 03/06/2026).

[3] Reynald Arnerin. *A journey in a procedural volume Optimization and filtering of Perlin noise*. 2009. URL: `https://maverick.inria.fr/Publications/2009/AN09/rapport_Reynald_Arnerin.pdf` (visited on 04/24/2026).

[4] Franky Hung. *Visualizing 3D Perlin Noise in Three.js*. 2023. URL: `https://franky-arkon-digital.medium.com/visualizing-3d-perlin-noise-in-three-js-1ade5ac9eb63` (visited on 03/06/2026).

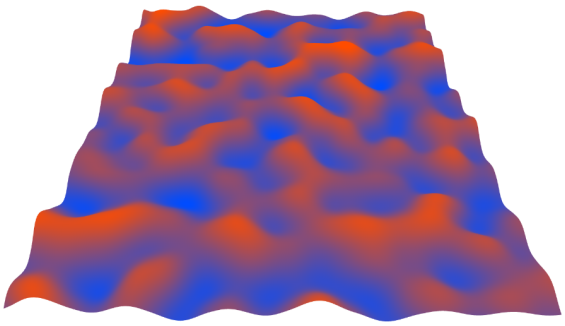
Images

(a) *Valeur seuil à* 0.4

(b) *Valeur seuil à* 0.6

Figure 1: *Surface 3D avec une valeur seuil fixée*

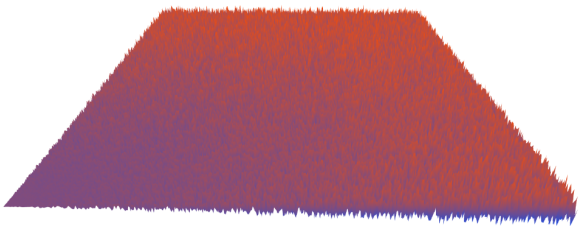
(a) *Fréquence* $\in \mathbb{N}$



(b) *Fréquence à* 0.02

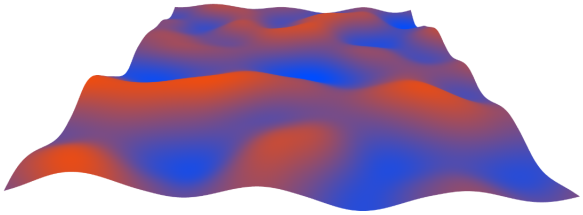
Figure 2: *Cas classiques de la fréquence*

(a) *Fréquence à* 0.001

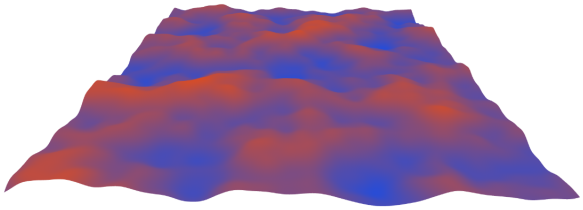


(b) *Fréquence à* 2.001

Figure 3: *Cas limites de la fréquence*

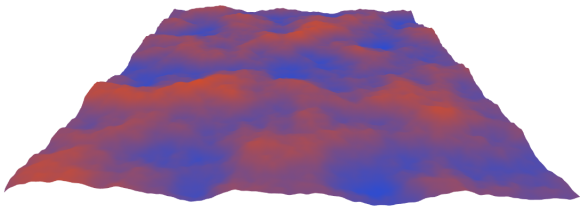


(a) *Surface avec une seule octave*

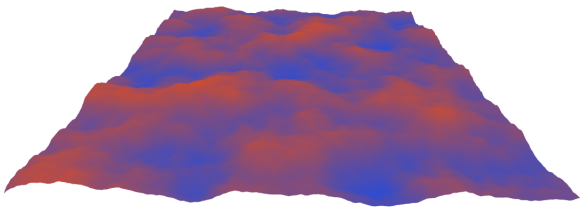


(b) *Surface avec trois octaves*

Figure 4: *Comparaison de surfaces avec les octaves*



(a) *Surface avec cinq octaves*



(b) *Surface avec six octaves*

Figure 5: *Saturation des octaves*